AssemblyFlow: Pipelined Execution Analyzer

Subject Micro Project Report: Design & Analysis of Algorithms (DAA)

Student Name:	Ashik Ameen	**Subject:**	Design & Analysis of Algorithms (DAA)
Project:	AssemblyFlow	**Academic Term:**	October 2026
Live Demo:	https://micro-project-a338.onrender.com/	**GitHub Repo:**	https://github.com/Ashik9500/Micro_project.git

1. Executive Summary & Problem Statement

In optimizing compilers and modern computer architectures, instruction scheduling determines an execution sequence that minimizes pipeline hazard stalls while strictly preserving data integrity. In **AssemblyFlow**, the **Design and Analysis of Algorithms (DAA)** module serves as **Stage 2**. It applies **Bernstein's Conditions for Concurrency** to extract register read/write footprints, constructs a **Directed Acyclic Graph (DAG)** with consolidated multi-hazard edges, and computes valid instruction orderings using **Kahn's Topological Sorting Algorithm** in optimal **O(V + E)** time with deterministic tie-breaking.

2. Theoretical Foundations in DAA

2.1 Register Usage Sets & Bernstein's Conditions

For each instruction I, the algorithm extracts its Read Set R(I) and Write Set W(I). For any pair I_i, I_j with i < j:
- **RAW (Read-After-Write / True Dependency):** $W(I_i) \cap R(I_j) \neq \emptyset$. I_j reads data written by I_i.
- **WAR (Write-After-Read / Anti-Dependency):** $R(I_i) \cap W(I_j) \neq \emptyset$. I_j overwrites data read by I_i.
- **WAW (Write-After-Write / Output Dependency):** $W(I_i) \cap W(I_j) \neq \emptyset$. Both instructions write to the same register.

2.2 Directed Acyclic Graph (DAG) Modeling & Edge Consolidation

Precedence constraints are modeled as a DAG G = (V, E), where V represents instruction nodes and E represents directed precedence edges. AssemblyFlow implements **Edge Consolidation**: when two instructions share multiple hazards (e.g., RAW and WAW simultaneously), they are represented by a single consolidated edge with composite annotations, avoiding redundant multi-edges.

2.3 Kahn's Algorithm for Topological Sorting

Kahn's algorithm operates on in-degrees ($deg^-(v)$):
1. Calculate in-degree for all vertices v in V.
2. Add all vertices with $deg^-(v) = 0$ to candidate set S.
3. While S is non-empty:
   - **Tie-Breaking:** Pick vertex u in S with lowest instruction index.
   - Append u to topological schedule L.
   - For each successor w of u: decrement $deg^-(w)$. If $deg^-(w) == 0$, insert w into S.
4. **Cycle Detection:** If |L| < |V|, a directed cycle exists, halting the pipeline.

3. Step-by-Step Kahn Trace Example

Instruction Stream: I1: LOAD R1, 10 | I2: ADD R2, R1, R3 | I3: SUB R4, R2, R5

Step	Candidate Set S	Selected Vertex	In-Degree Updates	Resulting Order L
1	{ I1 }	I1	I2 in-degree decremented: 1 → 0	[I1]
2	{ I2 }	I2	I3 in-degree decremented: 1 → 0	[I1, I2]
3	{ I3 }	I3	No successors remaining	[I1, I2, I3]

4. Asymptotic Complexity Analysis

Algorithm Phase	Time Complexity	Space Complexity	Analysis & Derivation
Read/Write Set Extraction	O(V)	O(V)	Fixed operand count (≤ 3) per instruction node
DAG Edge Construction	O(V^2)	O(V + E)	Pairwise Bernstein evaluation for all i < j
Kahn's Topological Sort	O(V + E)	O(V + E)	Each vertex and edge processed exactly once
Cycle Detection	O(1)	O(1)	Comparison of result list length |L| against |V|

5. Conclusions & Learning Outcomes

The DAA module bridges mathematical graph theory with practical compiler optimization. Students witness how abstract concepts like in-degrees, DAGs, Bernstein's concurrency conditions, and Kahn's algorithm directly determine valid execution schedules and prevent data corruption in pipelined processors.